

Independent
Internet-Draft
Intended status: Experimental
Expires: 11 March 2027

N. Perry
Independent
7 September 2026

Probabilistic Contact Metadata for DTN Bundle Routing
draft-perry-dtn-cpb-latest

Abstract

This document defines the Contact Probability Block (CPB), a BPv7 extension block carrying probability metadata between forwarding nodes in Delay/Disruption Tolerant Networks (DTNs). CPB provides a common transport for estimates produced and consumed by routing protocols such as PROPHET, CGR, MaxProp, RAPID, and Spray-and-Wait. Integration is optional and does not replace those algorithms.

CPB uses CBOR floating-point encoding, recommends IEEE 754 binary16 for bandwidth efficiency, preserves endpoint identifiers, and defines BPsec protection for authenticated routing use. Metric-type tags and shared applicability context distinguish estimates; cross-metric arithmetic is prohibited.

This Experimental draft outlines a plan to evaluate routing behavior, robustness to noisy or adversarial estimates, and interoperability across independent BPv7 implementations. It reports no experimental results or demonstrated routing-performance improvement.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 11 March 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	4
1.1. Problem Statement (IPNSIG SSI Architecture, Section 4.2)	4
1.2. Motivation	5
1.3. Requirements Language	6
1.4. Terminology	6
1.5. Notation Conventions	6
2. Architectural Considerations	7
2.1. Why NOT URI Query Parameters	7
2.1.1. Endpoint Identifier Immutability	7
2.1.2. Router Modification Problem	8
2.2. Solution: Bundle Extension Blocks	8
3. Contact Probability Block	9
3.1. Overview	9
3.2. Block Structure	9
3.2.1. Block Processing Flags	13
3.2.2. Cyclic Redundancy Check (CRC) Handling	15
3.2.3. Hop Count Block Co-Presence	15
3.3. Block Type Code	15
3.3.1. IANA Registration	15
3.3.2. Backwards Compatibility	16
3.4. CBOR Encoding of Probability Values	16
3.4.1. Invalid Float Handling	17
3.4.2. Encoding Details	18
3.4.3. Hex Encoding Table	18
3.5. Metric-Type Semantics and Cross-Metric Comparison	19
3.6. Multiple CPBs in One Bundle	22
3.6.1. Per-Path Matching Rules	23
4. Operational Semantics	24
4.1. Creation	24
4.2. Processing by Intermediate Routers	25
4.3. Usage in Routing Algorithms	26

4.4. Stability Considerations and Feedback Dynamics	26
5. Relationship to Existing DTN Routing Protocols	27
5.1. PROPHET (RFC 6693)	28
5.2. MaxProp	28
5.3. RAPID	28
5.4. Spray-and-Wait Family	29
5.5. Epidemic and Summary-Vector Protocols	29
5.6. Contact Graph Routing (CGR)	29
5.7. Social and Hybrid Metrics	30
5.8. DTNMA (RFC 9675)	30
5.9. Neighborhood Advertisement and Scheduled Topology	30
5.10. Summary: CPB as Cross-Cutting Metadata	31
6. Backwards Compatibility	31
7. Operational Considerations	31
7.1. When and Where to Emit CPBs	31
7.2. Bundle Store and Memory Sizing	32
7.3. Computational and Storage Overhead on Constrained Nodes	32
7.4. Logging, Monitoring, and OAM	32
7.5. Partial Deployment	33
7.6. Congestion and Storage Pressure	33
7.7. Clock Considerations	33
8. Security Considerations	33
8.1. Threat Model	33
8.2. Trust Model	34
8.2.1. Who is Authorized to Append CPBs	34
8.2.2. Cryptographic Adjudication of Conflicting CPBs	35
8.2.3. Downstream Handling of Upstream BIBs	35
8.2.4. Route Poisoning by Semi-Trusted Relays	36
8.2.5. Inter-Domain Trust	36
8.2.6. Replay Resistance	37
8.3. Integrity Protection	37
8.3.1. Source CPB with Append-Only Intermediates	38
8.3.2. Strip-and-Replace Model	38
8.3.3. Prohibition on BCB Encryption	38
8.4. Mitigations	38
8.5. Privacy Considerations	39
9. IANA Considerations	39
9.1. Guidance for Designated Experts	40
10. Overhead Analysis	40
10.1. Computational and Storage Overhead	42
11. Implementation Status	43
12. Planned Experimental Evaluation	44
13. Reference Implementation	44
13.1. Repository	45
13.2. Reference Implementation Development Checks	45
13.3. Relationship to Updates of BPV7	45
14. References	46

14.1. Normative References	46
14.2. Informative References	47
Appendix A. Mars Mission Scenario	49
Appendix B. Acknowledgements	50
Appendix C. Change Log	50
C.1. Unsubmitted working draft (September 2026)	50
Author's Address	51

1. Introduction

Delay-tolerant networks (DTNs) often face uncertain contact schedules, such as during Mars mission disruptions, where standard BPv7 routing like Contact Graph Routing (CGR) may delay delivery due to reliance on fixed contact plans. Probabilistic routing metadata enables adaptive forwarding decisions by communicating contact likelihood estimates between nodes.

This specification defines a bundle extension block (the Contact Probability Block, or CPB) for carrying probabilistic contact metadata that routing protocols may consult in DTNs, including applications such as space exploration communications. It does not report experimental results or claim demonstrated routing-performance improvement.

1.1. Problem Statement (IPNSIG SSI Architecture, Section 4.2)

The Interplanetary Networking Special Interest Group (IPNSIG), the Interplanetary Chapter of the Internet Society, published "Solar System Internet Architecture and Governance – from the Moon to Mars and beyond" in September 2023 [IPNSIG2023]. Section 4.2 ("Routing and Forwarding") of that document frames the routing problem this specification addresses:

| The routing and forwarding procedures of the terrestrial Internet
| would not adapt well in the Solar System Internet, as topological
| state information could not be provided in a timely manner due to
| the lengthy signal propagation delays, frequent lapses of
| connectivity, and dynamic topology... New technology is needed.

The first "Considerations" bullet of IPNSIG Section 4.2 directly invites the design approach taken by this specification:

| By what delay-tolerant mechanism(s) does this technology obtain
| the information on which it decides which next-hop node to forward
| a given bundle to? Does it do so by route computation, by scoring
| and/or preferential ranking of neighboring nodes, or in some other
| way?

The Contact Probability Block defined here is precisely such a "scoring and/or preferential ranking" signal, carried in-bundle as metadata for forwarding nodes to consult. The CPB design responds directly to two of IPNSIG Section 4.2's four numbered Recommendations:

Recommendation 1 (Autonomy and Automation): "routing and forwarding technologies are needed that do not require continuous human intervention or management from Earth." CPB enables autonomous, in-bundle probability annotation that each forwarder consults locally, without round-trips to Earth.

Recommendation 2 (Standards): "Autonomy, in turn, requires standard methods of: propagating whatever information is needed to support route computation; utilizing that information in common ways that assure coherent forwarding." CPB standardizes both the wire format for propagating probability information (Section 3.4) and the precedence and matching rules for utilizing it (Section 3.6, Section 4.2).

Section 5 positions CPB relative to the routing-protocol families catalogued in IPNSIG Section 4.2 and its Appendix A: PROPHET [RFC6693], Spray and Wait [Spyropoulos05], Schedule-Aware Bundle Routing (SABR) [SABR], Contact Graph Routing (CGR) [CGR], and Opportunistic CGR with Uncertain Contact Plans (CGR-UCoP) [Raverta21]. CPB is a cross-protocol metadata layer, not a replacement for any of these algorithms; each can consume or produce CPB data.

1.2. Motivation

Standard DTN routing protocols operate with deterministic contact schedules or historical encounter statistics. However, in highly disrupted networks:

- * Contact schedules change unpredictably (dust storms on Mars, solar weather, orbital variations)
- * Historical data becomes stale
- * Coordinated path computation elements (PCEs) can estimate probabilities but need a way to communicate them to forwarding nodes
- * Adaptive routing decisions require explicit probability metadata at the bundle level

By embedding probability estimates directly in bundles as a standard extension block, this specification gives DTN routing protocols an optional, common transport for those estimates. Whether any given protocol benefits is left to the planned experimental evaluation (Section 12); this document asserts no demonstrated improvement.

1.3. Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

1.4. Terminology

Contact Probability The likelihood of successful bundle delivery to a destination node via a particular next hop or path, expressed as a value in [0.0, 1.0].

Contact Probability Block (CPB) A bundle extension block that carries contact probability metadata, routing hints, and temporal context.

Path Probability A probability estimate specific to a particular next-hop neighbor or outgoing interface.

Default Probability An overall or baseline probability estimate when per-path probabilities are unavailable.

Metric Type An identifier distinguishing the semantic meaning of the probability value (e.g., PROPHET delivery predictability, CGR contact confidence, MaxProp delivery likelihood).

Validity Duration The time period (in seconds) after creation during which the probability estimate remains usable for routing decisions.

1.5. Notation Conventions

This document uses three distinct notations for describing data structures, following the conventions of other DTN/BPv7 documents:

CDDL (Concise Data Definition Language) Formal schema definitions per [RFC8610]. Listings are labeled "type: cddl". CDDL defines the structural constraints that implementations MUST conform to.

CBOR Extended Diagnostic Notation (EDN) Human-readable

representations of CBOR data per [RFC8610] Appendix G. Listings are labeled "type: cbor-diag". EDN examples show concrete data values in readable form.

Annotated Hex Binary Base-16 encoded wire format with inline comments. Listings are labeled "type: hex". These show the exact byte sequences that appear on the wire.

2. Architectural Considerations

This section explains the architectural rationale for using bundle extension blocks rather than URI query parameters for probabilistic routing metadata.

2.1. Why NOT URI Query Parameters

An earlier design considered encoding probability as a URI query parameter in the destination EID:

```
<CODE BEGINS>
ipn:100.1?prob=0.75
<CODE ENDS>
```

That construction is architecturally flawed: it violates fundamental DTN principles for the following reasons:

2.1.1. Endpoint Identifier Immutability

Endpoint identifiers (EIDs) are immutable addresses that uniquely identify bundle destinations. If routing metadata were embedded in EIDs:

- * Intermediate routers would need to modify destination addresses to update or strip probability parameters
- * This violates the end-to-end principle and the Bundle Protocol specification [RFC9171]
- * Routers cannot determine whether ipn:100.1?prob=0.75 and ipn:100.1?prob=0.5 represent the same or different endpoints
- * Destination nodes would receive bundles with varying EID formats depending on router processing
- * Security mechanisms binding bundles to specific EIDs (BPsec [RFC9172]) would break if routers modified destinations

- * EID-based duplicate detection would fail when the same logical endpoint has multiple representations
- * Bundle uniqueness and lifetime handling that depend on a stable destination EID would be disrupted

2.1.2. Router Modification Problem

If probability were in the destination EID, the following scenario occurs:

1. Router A receives a bundle destined to ipn:100.1?prob=0.75
2. Router A consults the probability for forwarding decisions
3. Router A must now decide:
 - * Forward as-is with ?prob=0.75 (but this value may be stale/incorrect for the next hop)
 - * Strip the parameter to ipn:100.1 (modifying the destination address)
 - * Update the parameter to a new value, e.g., ?prob=0.6 (also modifying the destination address)
4. Any modification violates the immutability principle and breaks downstream systems

This architectural violation is unacceptable in the DTN model.

2.2. Solution: Bundle Extension Blocks

This specification resolves the architectural conflict by carrying probabilistic metadata in bundle extension blocks, completely separate from endpoint identifiers:

- * Destination EID remains immutable and unchanged: ipn:100.1
- * Routing metadata travels in extension blocks (the Contact Probability Block)
- * Routers can freely add, remove, or modify metadata without affecting the destination EID
- * Security is maintained independently using Bundle Integrity Blocks (BIBs)

- * Legacy nodes safely ignore unknown extension blocks without affecting bundle delivery

This design maintains clean architectural separation between:

- * Addressing (EIDs): immutable, end-to-end
- * Routing Hints (CPB metadata): mutable, hop-by-hop

3. Contact Probability Block

3.1. Overview

The Contact Probability Block (CPB) is a standard BPv7 bundle extension block that carries contact probability metadata. Its purpose is to communicate probability estimates from source nodes, path computation elements (PCEs), or intermediate routers to downstream forwarding nodes.

3.2. Block Structure

The CPB has the standard bundle extension block structure defined in [RFC9171]. The canonical block structure is a CBOR array per RFC 9171 Section 4.3.2. The diagram shows field order schematically; CBOR fields have variable-length encodings, so its columns do not specify bit offsets or fixed field widths:

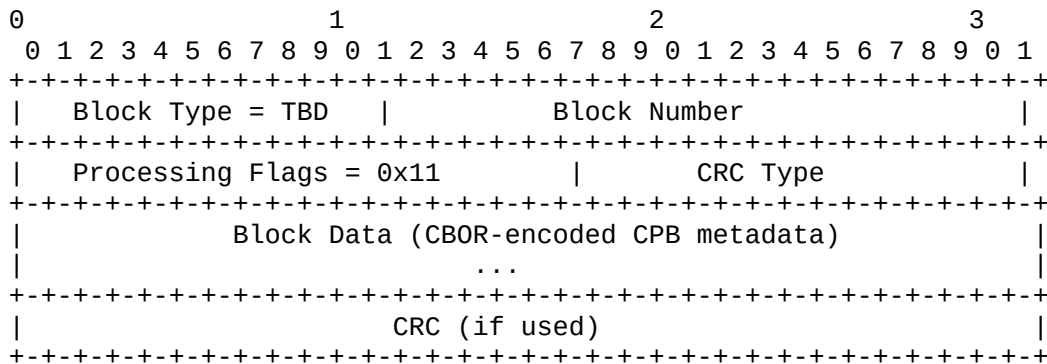


Figure 1: Bundle Extension Block Field Order (Schematic)

The block-type-specific data (BTSD) field is a CBOR byte string (bstr) whose content is a CBOR-encoded cpb-data map. This byte-string embedding follows RFC 9171's canonical block structure, where the BTSD type is bstr (not any). The formal CDDL binding follows below.

The following annotated hex shows a CPB with default probability 0.75 and a per-path probability of 1.0 for next-hop node 100. CRC type 0 omits the CRC. This example shows the canonical block alone; a surrounding bundle and any BIB are not shown:

```
<CODE BEGINS>
85                ; canonical block: array with 5 elements
18 C8             ; type=200 (RFC 9171 experimental range)
02               ; num=2
11               ; flags=0x11 (replicate; discard-if-unproc)
00               ; crc-type=0 (no CRC; CRC field omitted)
57               ; BTSD: bstr (23 bytes)
  A4              ; map with 4 entries
  00 F93A00       ;   0: prob=0.75 (float16)
  01 81           ;   1: array of 1 path entry
    82           ;     [node, prob]
    18 64         ;       node=100
    F93C00        ;       prob=1.0
  02 1A 322EC0C0 ;   2: timestamp (seconds)
    ;           ;   2026-09-05 12:00:00 UTC
  04 19 0E10     ;   4: validity=3600s
<CODE ENDS>
```

Figure 2: CPB Concrete Example (type: hex)

The CBOR data structure in the block-data field is defined as follows. The BTSD is byte-string-embedded per RFC 9171's canonical block structure, using the CDDL socket and generic rules from RFC 9171 Section 4.3.2:

```
<CODE BEGINS>
; Integration with RFC 9171 canonical block structure.
; The BTSD is a bstr whose content is CBOR-encoded cpb-data.
; Hop-by-hop CPB MUST stay readable; do not BCB-encrypt (Sec 8.3.3).
; Plain bstr alt = RFC 9171 embedded-cbor (opaque BTSD).

; 200 = experimental-range example used in this document
; (RFC 9171 192-255). Replace after IANA assignment.
$extension-block /=
  extension-block-use<200, (bstr .cbor cpb-data / bstr)>
<CODE ENDS>
```

Figure 3: CPB Extension Block Binding (type: cddl)

```

<CODE BEGINS>
cpb-data = {
    ? 0: probability,           ; Default/overall probability
    ? 1: [* path-entry],       ; per-path (SHOULD limit size; Sec 3.4)
    ? 2: uint,                 ; Timestamp (CPB epoch seconds)
    ? 3: bstr,                 ; Source PCE identifier (optional)
    ? 4: uint,                 ; Validity (seconds; requires field 2)
    ? 5: metric-type,          ; Metric type identifier
    ? 6: probability,          ; Profile-defined confidence
    ? 7: uint,                 ; format version (1 = this spec)
    * (int .gt 7) => any        ; Extensibility (future fields)
}

probability = float            ; See Section 3.4 for encoding

; At least one of field 0 or field 1 SHOULD be present for a
; usable estimate. An empty map is valid CBOR but no routing signal.

path-entry = [
    next-hop: eid-reference,    ; Next-hop node identifier
    probability: float          ; Probability for this path
]

eid-reference = uint / tstr     ; packed IPN FQNN or non-IPN EID text

metric-type = &(
    prophet-dp: 0,              ; PROPHET delivery predictability
    cgr-confidence: 1,          ; CGR contact confidence
    maxprop-delivery-likelihood: 2, ; MaxProp higher-is-better score
    rapid-utility: 3,           ; RAPID utility metric
    generic: 4,                 ; Unspecified/generic probability
) / uint                       ; Extensible via future assignment
<CODE ENDS>

```

Figure 4: CPB Data Structure (type: cddl)

**Note on bstr .cbor embedding:* The CDDL .cbor control operator ([RFC8610] Section 3.8.4) indicates that the byte string contains valid CBOR matching the cpb-data rule. The plain bstr alternative in the extension-block binding matches the RFC 9171 embedded-cbor pattern and permits an opaque byte string at the structural binding layer. It does not authorize encryption of a CPB: CPB content processing requires the cpb-data map, and the prohibition on BCB encryption in Section 8.3.3 applies.

**Field Semantics:*

Field 0: Default Probability An overall probability estimate when

per-path data is unavailable. Range [0.0, 1.0].

Field 1: Per-Path Array An array of [next-hop, probability] tuples.

Senders SHOULD NOT include more than 8 entries. Receivers on bandwidth- or memory-constrained links SHOULD enforce a local policy limit (8 entries is RECOMMENDED) and MAY treat a CPB exceeding the local limit as malformed for that deployment, logging under Section 7.4. The CDDL array is unbounded; the size recommendation is the prose requirement above. Receivers and senders on unconstrained links MAY use larger arrays.

Field 2: Timestamp Creation time of this CPB estimate, in whole seconds since 2000-01-01 00:00:00 UTC (the same epoch origin as DTN Time in [RFC9171], but this field is defined in *seconds*, not the millisecond DTN Time unit of RFC 9171 Section 4.2.6).

Receivers MUST interpret field 2 and field 4 in this same second-based unit. Enables receivers to compute age and apply local decay policies. Implementations MUST NOT copy an RFC 9171 DTN Time or Bundle Age value (milliseconds) into field 2 without converting to whole seconds (integer division by 1000). Example: DTN Time 841924800000 ms is CPB timestamp 841924800 (0x322EC0C0).

Field 3: Source PCE Identifier An opaque byte string identifying the entity (e.g., a PCE or router) that computed the probability values in this CPB. This field supports trust and policy decisions (see Section 8). No particular encoding or registry is defined. The value is meaningful only within the administrative scope(s) that trust the producer. The identifier is a claim of provenance. For authenticated routing use, the receiver MUST establish that the authenticated security principal is authorized to assert this identifier, including any configured delegation, as specified in Section 8.2.1. Deployments MAY omit this field when no source attribution is required or when an authenticated principal or explicit local policy supplies the required producer binding.

Field 4: Validity Duration (TTL) The number of seconds after the

timestamp during which this probability estimate remains usable. A sender including field 4 MUST also include field 2; a receiver MUST treat field 4 without field 2 as malformed. When the sum of timestamp and validity duration is less than the current time, the CPB is expired and MUST be ignored in full for routing (all probability fields, including per-path entries). If field 4 is omitted but field 2 is present, implementations SHOULD apply deployment-specific age thresholds (Section 4.1). If both fields are absent, the receiver MUST NOT infer a creation timestamp. It MAY consume the undated CPB only under an explicit local policy that sets a finite validity interval for its use; otherwise, it MUST treat the estimate as absent for routing.

Field 5: Metric Type Identifies the semantic meaning of the probability value. When absent, implementations MUST treat the value as generic (type 4). An unrecognized metric-type codepoint remains distinct: receivers MUST NOT coerce it to type 4 (see below).

Field 6: Confidence An optional float value in [0.0, 1.0] representing the confidence or certainty in the probability estimate. Its calibration and interpretation MUST be established by the shared deployment profile before a receiver compares or weights it, as described in Section 3.5. Observation count alone does not determine this value. Routing algorithms MAY use compatible values for risk-aware forwarding.

Field 7: Version CPB format version number. If absent or set to 1, the structure and semantics are as defined in this specification. Receivers SHOULD process all recognized fields when encountering a higher version number and MAY ignore unrecognized higher fields or additional semantics defined in future specifications. Senders using only the fields defined here SHOULD omit the version field or set it to 1.

Receivers encountering an unrecognized metric-type value SHOULD ignore that CPB for routing unless local policy explicitly maps the codepoint. They MUST NOT silently coerce an unknown metric-type into metric-type 4 (generic) for automated arithmetic or ranking. Senders MUST use only registered values defined in the IANA "CPB Metric Types" registry (see Section 9) unless operating in a private or experimental context.

3.2.1. Block Processing Flags

The CPB MUST set the block processing control flags ([RFC9171] Section 4.2.4) as follows:

- * *Bit 0 (0x01): Replicate in all fragments:* Implementers MUST set this bit so that if a bundle with a CPB is fragmented during transmission, each fragment carries a copy of the CPB.
- * *Bit 4 (0x10): Discard block if it can't be processed:* Implementers MUST set this bit because the CPB is advisory routing metadata, not critical to bundle delivery.
- * *Bit 1 (status report if unprocessable) and bit 2 (delete bundle if unprocessable):*MUST be 0. CPB-specific content errors MUST NOT request a status report or bundle deletion through these flags. Core BP and BPsec processing still applies (see Section 3.4.1).
- * *Bits 3, 5, and 6:*MUST be 0 (Reserved in [RFC9171]). *Bit 7 and higher unassigned bits:* MUST be 0 until assigned.

In practice, both flags set means the block processing flags byte is set to 0x11 (binary 00010001).

Path-entry handling under fragmentation: A fragmenting node MUST NOT modify a CPB covered by a BPsec security operation, including pruning entries or re-encoding its block-type-specific data. For a CPB without such protection, the node MAY prune path-entry elements (field 1) known to be unreachable in the fragment's remaining path, but MUST NOT alter the probability values of retained entries. The default-probability field (0) and metric-type field (5) MUST NOT be modified during fragmentation. If pruning changes a CPB with an attached CRC, the fragmenting node MUST recalculate that CRC.

CPB replication does not by itself replicate a protecting BIB. When creating a BIB whose sole target is a CPB, the security source SHOULD set the BIB's "replicate in every fragment" flag before computing its security result. Replication and processing of BIBs with additional targets follow [RFC9171] and [RFC9172] Section 5.2, taking all targets into account. Fragmentation MUST NOT change authenticated BIB header fields to arrange replication after the security result is computed.

The profile for authenticated routing in Section 8.3 binds a CPB to its containing bundle. Even if the CPB and BIB are both copied, changing the primary block for fragmentation prevents primary-bound verification before reassembly, as described in [RFC9173] Section 6.5. A router requiring authenticated CPB input MUST treat CPBs in fragments as unavailable for routing until reassembly permits verification, and use local routing state instead. A node MUST NOT add a BIB or BCB to a bundle marked as a fragment ([RFC9172] Section 5.2); the CPB update models in Section 8.3 do not override

this restriction. Deployments needing authenticated metadata on each fragment can use an appropriate encapsulation mechanism that applies security to a containing nonfragment bundle. This specification does not define such a mechanism.

3.2.2. Cyclic Redundancy Check (CRC) Handling

CRC Type and CRC are defined in [RFC9171] Sections 4.2.1 and 4.2.2. Their placement in the canonical bundle block array is specified in Section 4.3.2. The CRC is optional:

- * ***No CRC (recommended):*** Set `crc-type` to 0 ([RFC9171] Section 4.2.1) and omit the CRC field from the canonical block array (five elements total), per [RFC9171] Section 4.3.2. This is the default and minimizes overhead. Implementations **MUST NOT** emit a CBOR null CRC element when `crc-type` is 0.
- * ***Optional CRC:*** Implementations **MAY** include CRC type 1 (X.25 CRC-16) or type 2 (CRC32C / Castagnoli) as defined by [RFC9171] Sections 4.2.1 and 4.2.2.

Implementations **SHOULD** prefer BPSec BIBs (Section 8.3) over CRCs for integrity, using a context that provides cryptographic authentication as specified there. A CRC detects transmission errors and does not authenticate a producer.

3.2.3. Hop Count Block Co-Presence

Bundles carrying a CPB **SHOULD** also carry a Hop Count block (block type 10, [RFC9171] Section 4.4.3). The Hop Count block provides a loop termination guarantee independent of probability-based routing decisions.

3.3. Block Type Code

The bundle extension block format ([RFC9171]) requires each extension block to have a type code that identifies what kind of block it is.

3.3.1. IANA Registration

This specification requests IANA to assign a permanent block type code from the "Bundle Block Types" registry (created by [RFC9171]). Until IANA assignment:

- * Implementations **MAY** use values in the 192-255 range for experimental/private use ([RFC9171] Section 4.3.2)
- * The example hex dump uses code 200 (0xC8) for illustration

- * Production deployments SHOULD use the IANA-assigned code

3.3.2. Backwards Compatibility

Routers that do not recognize the CPB block type code process the canonical block as specified in [RFC9171] Section 4.2.4:

1. Read the block processing control flags (the third element of the canonical block array, not a fixed octet offset).
2. If bit 4 (0x10) is set, as this specification requires, discard the unrecognized CPB and continue the bundle.
3. There is no BPv7 flag meaning "remove if can't process" (that language is from BPv6). Do not invent one.

3.4. CBOR Encoding of Probability Values

The CDDL schema (Section 3.2) defines probability fields as float rather than constraining to a specific precision, because enforcing float16 in CDDL is difficult for encoders to implement in practice. Instead, the precision requirement is expressed as a prose recommendation:

Implementations SHOULD choose probability values that are exactly representable in IEEE 754 binary16 (half-precision). Binary16 provides nearby approximations for values in [0.0, 1.0]; decimal values such as 0.95 generally require rounding. The reference encoder selects the nearest binary16 value after applying the small-value policy below. When a value is representable in float16, [RFC8949] Section 4.2.1's deterministic encoding requirement ensures it will be encoded in 3 bytes.

- * ***Range:** [0.0, 1.0], sufficient for all probability values
- * ***Precision:** Approximately 3 decimal digits, adequate for routing decisions
- * ***Overhead:** 3 bytes per value when float16 is used
- * ***Standard:** Defined in [RFC8949] (CBOR specification)

Implementations that produce probability values not representable in float16 will incur 5 bytes (float32) or 9 bytes (float64) per value under deterministic encoding. Receivers MUST accept finite CBOR float encodings (binary16, binary32, or binary64). They MUST reject NaN and +/-Infinity as malformed, then MUST reject finite values outside [0.0, 1.0] as malformed. Clamping out-of-range values to the

interval endpoints is NOT RECOMMENDED because mapping a corrupted value greater than 1.0 to certainty can amplify sinkhole attraction (see Section 8).

***Bandwidth Considerations and Array Limits:** A CPB with per-path probabilities to multiple relays adds overhead (5 bytes per entry for a one-byte uint next hop and binary16 probability, increasing with the encoded next-hop length). To mitigate DoS risks from oversized arrays on low-bandwidth or constrained links, senders SHOULD NOT produce per-path arrays larger than 8 entries. Receivers in constrained deployments SHOULD enforce a local limit (default 8) and treat excess entries as malformed for that link, logging the event under Section 7.4. This guidance is normative for resource protection. The CDDL for field 1 is [* path-entry] (unbounded); the limit of 8 is prose, not a CDDL constraint.

A CPB BTSD MUST contain exactly one cpb-data map, with no trailing CBOR items or unused bytes. Its keys MUST be unique unsigned integers; duplicate keys make the CPB malformed, including duplicate unknown field keys. Receivers MUST detect duplicates before a generic decoder collapses map entries. Known fields MUST use the CBOR types specified by the CDDL. Booleans and tagged bignums are not uints; probability fields require untagged CBOR floats even when their values are exactly 0 or 1. A uint is in the range 0 through 18446744073709551615. Malformed CPB content is handled as specified in Section 3.4.1.

3.4.1. Invalid Float Handling

Implementations MUST handle invalid or out-of-range probability values as follows:

- * ***NaN (0xF97E00):** MUST be rejected as malformed.
- * ***+/-Infinity (0xF97C00, 0xF9FC00):** MUST be rejected as malformed.
- * ***Negative values:** MUST be rejected as malformed.
- * ***Values > 1.0:** MUST be rejected as malformed.
- * ***Very small positive values:** Decoded probabilities strictly between 0 and 2^{-14} (the smallest normal binary16 value) SHOULD be flushed to zero for embedded FPU compatibility. This threshold applies regardless of the received floating-point width.
- * ***Malformed CBOR:** Discard the CPB block and continue processing the bundle.

In all error cases, implementations SHOULD log the error (including source, timestamp, and offending value) for debugging. The discard-and-continue behavior above applies to CPB-specific parsing or value errors inside a well-formed canonical block and BTSD byte string. CPB processing MUST NOT request bundle deletion for those errors. Core BP reception checks, including malformed canonical blocks and CRC failures ([RFC9171] Section 5.6), and applicable BPsec security-policy processing remain in effect. Removal of a CPB and its associated security operations follows [RFC9172].

3.4.2. Encoding Details

When a probability value is representable in half-precision, the CBOR encoding uses major type 7 with additional information value 25 (16-bit IEEE 754 binary16):

Major Type: 7 (floating-point or simple)
 Additional Info: 25 (16-bit IEEE 754 float)
 Total Bytes: 3 (tag byte 0xF9 + 2 float bytes)

Figure 5: CBOR Encoding: Major Type 7, Additional Info 25

3.4.3. Hex Encoding Table

Probability	Binary16	CBOR Hex	Bytes
0.0	0x0000	0xF90000	3
0.25	0x3400	0xF93400	3
0.5	0x3800	0xF93800	3
0.75	0x3A00	0xF93A00	3
0.95	0x3B9A	0xF93B9A	3
1.0	0x3C00	0xF93C00	3

Table 1: CBOR Encodings for Common Probability Values

The 0.95 row denotes a nominal input: its nearest binary16 value, encoded as 0x3B9A, is 0.9501953125. The other listed probabilities are exactly representable.

3.5. Metric-Type Semantics and Cross-Metric Comparison

The probability values carried in a CPB are dimensionless numbers in $[0.0, 1.0]$, but their `_semantics_` differ substantially across routing protocols. The metric-type field (field 5, Section 3.2) distinguishes these semantics. This subsection specifies the constraints on combining values across metric types.

The defined metric types carry the following distinct semantics:

metric-type 0 (prophet-dp) PROPHET delivery predictability $P_{\text{A,B}}$ as defined in [RFC6693] Section 2.1.2: an empirical encounter-history statistic in $[0.0, 1.0]$. Reflects observed past co-location behavior (with aging and optional transitivity applied `_before_` encoding), not a scheduled-contact success probability. Values in a CPB MUST already be aged at the producing node (Equation 2 of RFC 6693); receivers MUST NOT re-apply a second aging constant unless they recompute predictabilities from their own encounter base. Default probability (field 0) represents the producer's $P_{\text{(producer, dest)}}$ toward the bundle destination. When field 1 is present, each path entry for next-hop NH MUST carry the producer's estimate of $P_{\text{(NH, dest)}}$ (the delivery predictability of that next hop toward the destination), not a repeat of $P_{\text{(producer, dest)}}$ (see Section 5.1). A consumer of field 0 MUST establish which PROPHET node is the producer under the deployment's identity and trust policy, and MUST NOT substitute that value for an unmatched next hop's predictability. An absent next-hop entry supplies no estimate for that next hop, unless it is the identified producer itself.

metric-type 1 (cgr-confidence) Per-contact or per-next-hop success probability against a known contact schedule (or contact-plan edge). Reflects forward-looking confidence that a planned contact occurs as scheduled (link availability, weather, pointing). Contact start/end times, one-way light time, and residual volume remain in the contact plan, not in CPB; CPB carries only the scalar confidence the plan or local estimator associates with a hop or default path. Before importing such a value, the deployment MUST establish a documented applicability profile that identifies the transmitting node, next hop, direction, and contact interval to which it applies, or a defined aggregate over identified contacts. The profile MAY obtain this context from a shared contact plan and the producer identity established by local policy; CPB does not encode that context itself. The computing entity need not be the transmitting node. Field 2 records estimate creation and field 4 limits estimate validity; neither identifies a contact interval. A consumer MUST NOT infer that an upstream estimate describes its own outgoing contact solely

because the next-hop identifier matches. If the consumer cannot establish the applicable context for a value, it MUST treat that value as absent for routing.

metric-type 2 (maxprop-delivery-likelihood) Higher-is-better delivery-likelihood proxy derived from MaxProp-style path cost. Producers MUST map lower MaxProp cost to a higher CPB value in $[0.0, 1.0]$ using a documented local inversion (for example, $1/(1+\text{cost})$ or a rank-normalized score). The wire value is not a raw cost. The inversion is lossy; original cost magnitude and multi-hop cost vectors are not recoverable from the CPB value. Hop-lists used for loop avoidance and delete-ACKs remain local MaxProp state, not CPB fields.

metric-type 3 (rapid-utility) Normalized per-packet utility from a RAPID-style optimizer, scaled by the producer into $[0.0, 1.0]$. This is an optimization-derived score (often over delay or resource constraints), not a calibrated delivery probability. Unnormalized utilities, delay distributions, and resource inventories are out of band; if a deployment cannot normalize utility into $[0.0, 1.0]$ honestly, it SHOULD register a new metric-type or use metric-type 4 only as advisory.

metric-type 4 (generic) Unspecified semantic. The producer makes no claim beyond "value in $[0.0, 1.0]$ ". Consumers SHOULD NOT assume any specific interpretation. Suitable for experimental social or hybrid scores (centrality, similarity) until a dedicated metric-type is registered.

Comparability within a metric type: Sharing a metric-type code does not by itself make values numerically comparable. Before comparing or combining values from different producers, bundles, or reference sets, consumers MUST establish a common interpretation through a documented deployment profile. For metric types 2 and 3, that profile specifies the cost or utility objective, normalization function and parameters, and the reference set over which scores are normalized. Rank-normalized values are only ordinal within that reference set unless the profile defines a common scale. Consumers MUST NOT apply shared numerical thresholds, cross-context magnitude comparisons, or arithmetic to values whose scales have not been established as compatible. Producer-relative ranking within its defined reference set remains possible. The same requirement applies to comparisons or weighting using confidence field 6; this specification defines no universal calibration for that field. Profiles are deployment context and are not signaled by the metric-type code alone.

Cross-metric arithmetic is prohibited. A router MUST NOT combine values from different metric types via arithmetic (addition, multiplication, weighted averaging, etc.) and treat the result as a meaningful probability. A PROPHET predictability of 0.7 and a CGR confidence of 0.7 are not interchangeable inputs to the same routing decision. Specifically:

- * A router using contact-confidence values in a confidence-weighted policy, optionally together with local contact rates, MUST use only metric-type 1 (cgr-confidence) values, and MUST treat path-entries of other metric types as if absent.
- * A router applying PROPHET-style transitivity rules MUST use only metric-type 0 (prophet-dp) values, and similarly ignore others.
- * A router consuming metric-type 4 (generic) SHOULD treat the value as advisory only and MUST NOT use it as the sole input to a routing decision.

Mixed-metric CPBs: A single CPB instance carries one metric-type (field 5). A bundle MAY carry multiple CPBs with different metric types simultaneously (for example, a CPB with cgr-confidence from a PCE and a separate CPB with prophet-dp accumulated at intermediate hops). The precedence rules of Section 3.6 apply within each compatible metric and applicability group; across metric types, each CPB is consumed only by routers implementing that metric's algorithm. A router that does not understand a particular metric type MUST ignore that CPB and continue processing.

Metric-type registry policy: Future routing protocols defining new probability semantics SHOULD register a new metric-type code (see Section 9) rather than overloading an existing one. The CPB Metric Types registry exists precisely to keep semantically distinct values separable on the wire.

Calibration and normalization: This document does not specify normalization between metric types because cross-metric equivalence has no rigorous theoretical foundation in the published DTN literature. Implementations that wish to combine evidence from multiple metrics for ranking purposes SHOULD do so using a documented, deployment-specific policy and SHOULD NOT rely on numerical comparability across types. This remains an open research question.

3.6. Multiple CPBs in One Bundle

A bundle may accumulate multiple Contact Probability Blocks as it traverses the network. Before applying precedence, a receiver **MUST** exclude CPBs that are malformed, expired, outside the accepted timestamp limits, or ineligible under its authorization and authentication policy. It **MUST** also exclude CPBs whose metric or applicability context it cannot interpret for the routing operation. Undated CPBs are eligible only under the explicit policy described for field 4.

The receiver **MUST** group eligible CPBs by metric type, applicability scope, and compatible normalization profile before resolving conflicts. Different metric types are never competitors in this selection. Within a group, differences in which path entries are present do not by themselves define a new group. Implementations **SHOULD** select one complete CPB per group using the following deterministic precedence:

For deployments that require authentication or otherwise apply a trust policy to CPB producers, implementations **MUST** use the following precedence within each eligible group (after excluding ineligible CPBs per above and Section 8.2.2):

1. Prefer the source ranked highest by the implementation's trust policy (authenticated and authorized producers before lower-ranked or unauthenticated sources that local policy still admits).
2. If trust ranks are equal, prefer a CPB with a timestamp over an undated CPB. Among dated CPBs at that trust rank, prefer the highest timestamp.
3. If still tied, prefer the highest bundle block number.

Deployments that intentionally consume unauthenticated CPBs and do not apply producer trust ranks **MAY** use timestamp-first ordering instead, but **SHOULD NOT** do so where route poisoning is a concern (Section 8.2.4).

Selection applies to the whole CPB, not independently to each field or path entry. A consumer MUST NOT fill gaps in the selected CPB with fields from superseded CPBs in that group. It applies Section 3.6.1 to the selected block and falls back to local routing state when that block supplies no applicable value. If there is no eligible CPB for a routing operation, the receiver MUST continue using local routing state. A deployment using an alternative precedence policy MUST document it and preserve the eligibility, compatibility, and whole-block selection rules.

```
<CODE BEGINS>
/ CBOR Extended Diagnostic Notation (EDN) /
{
    0: 0.75,                / Default probability /
    1: [
        [300, 0.5],        / To relay node 300: 0.5 /
        [100, 1.0]         / To orbiter node 100: 1.0 /
    ],
    5: 1                    / metric-type: cgr-confidence /
}
<CODE ENDS>
```

Figure 6: CPB with Per-Path Probabilities (type: cbor-diag)

```
<CODE BEGINS>
A3                      ; map with 3 entries
  00 F93A00             ; key 0: default prob = 0.75
  01 82                ; key 1: array with 2 path entries
    82                 ; path entry [node, prob]
    19 012C F93800     ;   node 300, prob 0.5
    82                 ; path entry [node, prob]
    18 64 F93C00       ;   node 100, prob 1.0
  05 01                ; key 5: metric-type = cgr-confidence
<CODE ENDS>
```

Figure 7: CPB Per-Path Wire Encoding (type: hex)

3.6.1. Per-Path Matching Rules

After eligibility and selection under Section 3.6, a router consults a selected CPB as follows:

1. Match the next-hop node/EID against path-entry keys using the scheme's identity comparison rules.
2. If a match is found and its metric-specific applicability context is established, use that probability.

3. Otherwise, use default probability (field 0) only when its metric-specific meaning and applicability profile cover this routing operation. In particular, metric 0's default describes the identified producer, and metric 1's default is limited to its defined contacts or aggregate.
4. If neither an entry nor a default applies, treat the estimate as absent and fall back to local routing state (do not invent a universal 0.5).

Senders **MUST NOT** include more than one path entry for the same next-hop identity. Receivers **MUST** treat such a CPB as malformed, including when the repeated entries have equal values. This check **MUST** use the same scheme-specific identity normalization as matching; comparing encoded strings alone is insufficient when the scheme permits equivalent representations. Identity normalization is for comparison only and does not authorize modification of a received CPB's authenticated bytes.

Each path-entry next-hop is encoded in that next-hop's own URI scheme. Path-entry schemes need not match the bundle destination EID scheme. For an IPN next-hop, the reference **MUST** be a CBOR unsigned integer encoding the Fully Qualified Node Number (FQNN) defined in [RFC9758] Section 6.1.1: (allocator_identifier << 32) | node_number. Each component is in the range 0 through $2^{32} - 1$. The LocalNode value (allocator 0, node number $2^{32} - 1$) **MUST NOT** be transmitted in a CPB. The service number is not part of an IPN next-hop key. Matching **MUST** compare decoded allocator and node components against the local next-hop identity, treating equivalent two- and three-component textual forms and binary encodings as the same identity. For example, ipn:977000.100.1 has next-hop key 0x000EE86800000064, whereas ipn:100.1 has next-hop key 100. A text-string reference **MUST** contain a full EID using a non-IPN URI scheme. Implementations **MUST NOT** encode an IPN next-hop as a text-string EID.

***Scheme Mismatch Handling:** When the next-hop scheme does not match the CPB's path-entry scheme, implementations apply the same metric-specific default test above; if no applicable default exists, they treat the estimate as absent and use local routing state. Implementations **SHOULD** log the scheme mismatch under Section 7.4.

4. Operational Semantics

4.1. Creation

The Contact Probability Block is created by:

- * The source application (using historical or predicted contact data)
- * A path computation element (PCE) (using network-wide routing state)
- * The first-hop router (using local contact statistics)

Implementations SHOULD include a timestamp (field 2) and a validity duration (field 4). Timestamps MUST be expressed in whole seconds since 2000-01-01 00:00:00 UTC (same epoch origin as DTN Time in [RFC9171] Section 4.2.6, but in seconds rather than the millisecond DTN Time unit; see field 2). Field 4 uses the same second-based unit so that timestamp + validity comparisons are dimensionally consistent.

In networks with significant propagation delays, clock synchronization is challenging. When validating a CPB timestamp, implementations SHOULD apply tolerance for clock drift and light-time delays:

- * **Forward skew tolerance:* Apply a configurable drift threshold (default: +/-600 seconds). CPBs with timestamps > current time + drift threshold MUST be discarded.
- * **Backward skew/age threshold:* CPBs with timestamps < current time - max_age MUST be discarded (default max_age: 86400s interplanetary, 3600s terrestrial).
- * **Deployment-specific adjustment:* Networks SHOULD define explicit max_drift and max_age per link characteristics.

4.2. Processing by Intermediate Routers

When a router processes a bundle containing a CPB:

1. **Read:* Apply the eligibility and selection rules in Section 3.6, then consult only the applicable default or per-path values under Section 3.6.1.
2. **Decide:* Use the probability as an input to routing decisions.
3. **Update (optional):* If the router has fresher probability data, it SHOULD append a new CPB block subject to the fragmentation and security-operation constraints in Section 3.2.1 and Section 8.3. Appending a CPB preserves integrity protection of an unchanged original CPB.

4. *Forward:* Transmit the bundle with the CPB(s) to the next hop.

4.3. Usage in Routing Algorithms

Routing algorithms MAY use the CPB in several ways:

- * *Local state preference:* Nodes that maintain native protocol state (PROPHET RIB, CGR contact plan, MaxProp tables) SHOULD prefer that local state over foreign CPB values unless local policy explicitly imports CPB as authoritative for a domain or metric-type.
- * *Threshold-based:* Forward if prob > threshold
- * *Weighted selection:* Choose proportionally to probability
- * *Replication target ranking:* When a local spray/epidemic policy already owns a remaining copy count L, CPB probabilities MAY rank candidate relays. CPB does not encode L and MUST NOT be read as defining fanout as $\text{ceil}(1/\text{prob})$ (see Section 5.4).
- * *Priority:* Prioritize higher-probability bundles during congestion
- * *Risk-aware:* Weight decisions by both probability and confidence (field 6)

4.4. Stability Considerations and Feedback Dynamics

Probability-aware forwarding creates feedback between routing decisions and observed network behavior. Naive use of CPB data can produce instability that confidence-blind routing avoids. Three dynamics are particularly relevant:

Herding When multiple sources receive identical high-confidence CPB advertisements for the same relay, they all forward to that relay simultaneously. The relay congests, its actual delivery probability drops, but in-flight CPBs continue advertising the stale high confidence. Traffic remains herded until updated CPBs propagate, which in DTN timeframes can take hours.

Route oscillation If a router switches preferences whenever CPB values cross a threshold, small probability fluctuations near the threshold cause traffic to oscillate between routes. Each switch may itself generate new CPB observations that further perturb the values.

Synchronized congestion collapse In dense topologies, simultaneous

reaction by many routers to the same CPB update can produce correlated congestion events. This is structurally similar to TCP synchronization in IP networks but with much longer recovery cycles due to DTN propagation delays.

Implementations SHOULD mitigate these dynamics. The following techniques are RECOMMENDED:

- * ***Hysteresis:** Use distinct enter and exit thresholds (e.g., switch `_to_` a route at $p > 0.75$, switch `_away_` only at $p < 0.60$). Avoids threshold-crossing chatter on small fluctuations.
- * ***Rate limiting:** Cap the frequency of routing preference changes per (source, destination) pair (e.g., at most one switch per contact opportunity, not per bundle).
- * ***Randomized tie-breaking:** When multiple routes have CPB probabilities within a configurable epsilon (default: 0.05), select randomly rather than deterministically by lowest EID. Breaks herding without sacrificing the high-probability choice.
- * ***Probability decay under load:** Maintain a local estimate of congestion on each next-hop and apply a load-aware discount to the received CPB probability before using it in routing decisions. Closes the feedback loop locally rather than waiting for upstream CPB refresh.
- * ***Bounded credulity:** Cap how heavily any single CPB can influence the local routing decision relative to the router's own observations. Specific weights are deployment-dependent.

A formal control-theoretic stability analysis of CPB-driven feedback dynamics is not provided in this document; this remains an open research question. Implementations that omit all of the above mitigations SHOULD NOT be deployed in production DTNs where congestion or herding could degrade service.

5. Relationship to Existing DTN Routing Protocols

This section positions CPB relative to the existing DTN routing-protocol families catalogued in IPNSIG Section 4.2 and its Appendix A ("Routing and Forwarding State of the Art").

5.1. PROPHET (RFC 6693)

PROPHET [RFC6693] is an epidemic protocol with strict pruning. Each node maintains delivery predictabilities $P_{(A,B)}$ updated on encounters (Eq. 1), aged over time (Eq. 2), and optionally improved by transitivity via an intermediate (Eq. 3). During a contact, peers exchange a Routing Information Base (RIB) of destination-scoped predictabilities, then apply a local forwarding strategy (for example GRTR / GTMX) and queueing policy. Positive delivery acknowledgments prune residual copies.

**What CPB carries for PROPHET:* metric-type 0 values that a producer has already computed (aged, optionally transitive) for the bundle destination and/or candidate next hops. CPB is an optional in-bundle hint so downstream nodes that do not speak the full PROPHET RIB exchange can still see a snapshot of the producer's P.

**What stays local (not CPB):* aging constants (γ , K), $P_{\text{encounter}}$ / beta parameters, the full RIB dictionary, Hello / anti-entropy session state, bundle offer and response TLVs, custody policy, and named forwarding/queueing strategies. CPB does not replace the PROPHET control plane in RFC 6693 Sections 3-5.

5.2. MaxProp

MaxProp estimates delivery likelihood from encounter history, prefers low path-cost relays, prioritizes new packets by hop count, uses hop-lists to avoid cycles, and floods delete-ACKs so peers can drop delivered bundles. Buffer drop order is tightly coupled to the same likelihood metric used for forwarding.

**What CPB carries:* metric-type 2 as a lossy [0.0, 1.0] higher-is-better score obtained by inverting MaxProp path cost into a delivery-likelihood proxy so foreign nodes can rank next hops without MaxProp's full peer exchange.

**What stays local:* cost vectors, hop-lists, delete-ACK flooding, and the dual use of likelihood for queue-drop priority (though a CPB-aware node MAY use metric-type 2 values as one input to local drop ranking; see Section 5.10).

5.3. RAPID

RAPID treats routing as a resource allocation problem: it computes a per-packet utility (often over expected delay under bandwidth and storage constraints) and replicates in decreasing marginal-utility order. Peers may exchange delay samples or related metadata to refine utilities.

What CPB carries: metric-type 3 only after the producer normalizes utility into [0.0, 1.0]. CPB does not carry raw delay distributions, residual bandwidth, or multi-objective utility vectors.

5.4. Spray-and-Wait Family

Spray-and-Wait [Spyropoulos05] limits replication to L copies. Binary spraying and Spray-and-Focus variants choose relays uniformly or by a utility in the focus phase. The remaining copy count is the primary control state.

What CPB carries: optional ranking hints (metric-type 0/1/2/3/4) for `_whom_` to spray to when a node chooses non-uniform targets.

What stays local / out of scope for CPB: the integer L remaining-copy counter and spray-phase state. Those are replication-control metadata, not probabilities; they need a different extension or local header if standardized later. CPB does not encode L.

5.5. Epidemic and Summary-Vector Protocols

Classic epidemic routing and many DTN anti-entropy designs exchange summary vectors of buffered bundle IDs so peers can request missing copies. That inventory exchange is orthogonal to probability metadata. CPB neither replaces summary vectors nor carries bundle-set digests. A node may still attach a CPB when forwarding a copy so receivers learn the forwarder's confidence snapshot.

5.6. Contact Graph Routing (CGR)

Contact Graph Routing and SABR-style schedule-aware routing compute earliest-arrival (or related) routes from a contact plan: contact intervals, data rates, one-way light times, and residual volume. CGR-UCoP [Raverta21] extends CGR using route delivery probabilities derived from Routing under Uncertain Contact Plans (RUCoP). Its decision model is distinct from the illustrative latency/confidence helper shipped with this draft.

What CPB carries: metric-type 1 confidences for default or per-next-hop edges whose transmitter and contact interval are established by the shared applicability context in Section 3.5. This lets scoped in-bundle state move with traffic when contact-plan dissemination is slow or partial.

What stays in the contact plan: time windows, rates, OWLT, and residual capacity. CPB is not a substitute contact plan. The closest architectural precedent for in-band routing metadata is the CGR Extension Block [CGREB]. This document does not compare forwarding policies or report routing-performance results; CGR-UCoP is prior art for simulation, not the definition of the block.

5.7. Social and Hybrid Metrics

Protocols such as BubbleRap, SimBet, and related social DTN routers use centrality, community labels, or similarity rather than pure encounter probability. Those scores may be published as metric-type 4 (generic) with local documentation, or as new metric-type codepoints via the IANA registry in Section 9. CPB still forbids mixing them arithmetically with prophet-dp or cgr-confidence.

5.8. DTNMA (RFC 9675)

[RFC9675] defines a DTN management architecture orthogonal to routing. A DTNMA agent could monitor CPB-related metrics (frequency of stale CPBs, scheme mismatches, BIB verification failures) for network health assessment.

5.9. Neighborhood Advertisement and Scheduled Topology

Secure Advertisement and Neighborhood Discovery (SAND) [I-D.ietf-dtn-bp-sand] is a DTN Working Group control-plane protocol. Nodes advertise local topology and short-horizon SABR-style metrics (data rate, one-way delay, bit-error ratio, and validity schedules) to topological neighbors. Those advertisements are not carried as metadata on application bundles.

CPB is complementary, not a substitute for SAND. Contact intervals, rates, OWLT, and residual volume remain in the contact plan or in SAND/SABR inputs. CPB carries optional in-bundle quality or confidence samples (metric-type tagged) along the data path when those estimates should travel with the bundle. A node MAY consume SAND metrics and CPB values together; it MUST NOT treat a SAND rate or delay advertisement as a CPB metric-type value, or vice versa.

Long-lived contact schedules and bandwidth as managed objects are modeled in Time-Variant Routing work, including the scheduled-attributes YANG model [I-D.ietf-tvr-schedule-yang] and related contact-plan YANG. Inter-domain reachability with optional time windows is the subject of the DTN Peering Protocol [I-D.taylor-dtn-dpp]. None of those documents define the CPB extension block.

5.10. Summary: CPB as Cross-Cutting Metadata

Across the families above, a recurring split appears:

- * **In-bundle scalar quality (CPB):** a [0.0, 1.0] score with an explicit metric-type, optional per-next-hop list, timestamp, and validity.
- * **Control-plane state (not CPB):** encounter RIBs, contact plans, summary vectors, copy counts, hop-lists, delete-ACKs, Hello sessions, and strategy/queue policy parameters.

The Contact Probability Block is designed as that cross-protocol scalar layer rather than a replacement for any algorithm. Algorithms MAY consume CPB for forwarding rank, replication target choice, or buffer-drop priority, and MAY produce CPB for downstream nodes, without requiring the peer to implement the producer's full protocol.

6. Backwards Compatibility

- * Nodes that do not recognize the CPB block type discard it under the required block processing flags and continue processing the bundle as specified in Section 3.3.2
- * The bundle's destination, source, and other standard fields are unaffected by the CPB
- * Legacy routing protocols (CGR, PROPHET, MaxProp, RAPID, Spray-and-Wait) continue to function without the CPB
- * The CPB is optional; bundles without it use standard routing behavior

7. Operational Considerations

This section collects deployment, sizing, and operational guidance for CPB. It draws from the processing rules in Section 4, the overhead analysis in Section 10, and the backwards-compatibility rules in Section 6.

7.1. When and Where to Emit CPBs

A CPB may be created by the source application (using historical or predicted contact data), a path computation element (PCE) (using network-wide state), or the first-hop router (using local contact statistics). Deployments SHOULD document their policy for which entity is authoritative for a given bundle class.

In environments with significant propagation delay (e.g., interplanetary), a PCE close to the source is often preferable so that the initial CPB reflects the most current available model. In low-delay terrestrial meshes, the first-hop router may have fresher local observations.

7.2. Bundle Store and Memory Sizing

If the strip-and-replace optimization in Section 8.3.2 is not used, bundles can accumulate multiple CPBs across hops. CPB size depends on the included fields and identifier lengths; the examples in Section 10 range from 12 to 100 bytes per canonical block before BIB overhead. These are illustrative configurations, not size bounds. Implementations on memory-constrained nodes SHOULD bound the number of retained CPBs per bundle and apply the removal of earlier instances, when authorized under Section 8.3.2, when appending a fresher CPB.

Bundle stores SHOULD account for the additional per-bundle overhead when dimensioning for CPB-enabled traffic.

7.3. Computational and Storage Overhead on Constrained Nodes

Decode and storage costs depend on the included fields, identifier lengths, implementation, and hardware. Section 10 gives encoded-size examples; this document provides no measured processing-time claims.

If the CPB is protected by a BPSec BIB, each consumption requires a signature or MAC verification. Implementations MAY cache verification results subject to the input and policy invalidation rules in Section 10.1.

Route recomputation triggered by new CPB observations can be bounded by the hysteresis and rate-limiting techniques described in Section 4.4.

Implementations SHOULD publish concrete profiling numbers (CPU cycles per CPB, memory delta, etc.) alongside interoperability reports, per the guidance in [RFC7942].

7.4. Logging, Monitoring, and OAM

Nodes SHOULD log CPB-related events (reception of invalid floats, scheme mismatches, version numbers higher than supported, BIB verification failures on CPBs, etc.) using deployment-defined management procedures. The adaptive information push in [RFC9675] Section 4.3 is architectural guidance; neither that section nor CPB defines a CPB event schema or logging API. Logging SHOULD be rate

limited so malformed input cannot exhaust management resources.

When a CPB is discarded due to age, future-dated timestamp, or malformed content, a management event **SHOULD** be generated to aid debugging of clock skew or misbehaving producers.

7.5. Partial Deployment

CPB is designed for incremental deployment. Nodes that do not recognize the block type discard the CPB under its required processing flags and continue forwarding the bundle (see Section 6). CPB metadata therefore does not cross a legacy node unchanged. Deployments **SHOULD** monitor the fraction of CPB-aware nodes and may choose to emit CPBs only on segments where a useful number of consumers exist.

7.6. Congestion and Storage Pressure

Probability-aware routing tends to concentrate traffic on high-confidence routes. Under sustained load this can lead to congestion on preferred paths and storage pressure at intermediate nodes.

The mitigations in Section 4.4 (hysteresis on route changes, minimum interval between CPB-triggered recomputations, and explicit rate limiting) are **RECOMMENDED**. Operators **SHOULD** monitor queue depths and delivery ratios on high-confidence routes and be prepared to fall back to confidence-blind policies or to reduce CPB production rate during overload.

7.7. Clock Considerations

CPB timestamps (field 2) are whole seconds since the DTN Epoch origin; validity durations (field 4) are elapsed seconds from that timestamp. Neither uses RFC 9171's millisecond DTN Time unit. In networks with light-time delay or unsynchronized clocks, the age and future-dated checks in Section 4.1 must use deployment-specific drift and max_age values. A deployment may need larger tolerances (e.g., +/- 600 s drift, 86400 s max age) than another; these are examples, not mission-independent defaults.

8. Security Considerations

8.1. Threat Model

- * ***Untrusted networks:** An attacker could inject false probability values to cause mis-routing (classic blackhole/sinkhole attraction via `_inflated_` metrics that draw traffic then drop it; selfish avoidance via `deflated` metrics)

- * ***Stale data:** A captured bundle with outdated probabilities could cause poor routing decisions
- * ***Inference attacks:** Observing probability values over time could reveal traffic patterns or network topology
- * ***Route poisoning by semi-trusted relays:** A relay that holds valid credentials within an administrative domain may still produce CPB advertisements that systematically bias routing decisions toward itself, away from a competitor, or against specific destinations
- * ***Replay of stale CPBs:** An attacker captures a recent valid (BIB-signed) CPB and replays it later when the underlying conditions have changed
- * ***Inter-domain trust escalation:** A CPB created by an entity trusted in one administrative domain crosses into a second domain where the producer is unknown; the second domain has no basis for trust evaluation

8.2. Trust Model

This document does not mandate a single trust model because DTN deployments span a wide range of administrative scenarios – from a single-operator interplanetary relay network to multi-party terrestrial mesh deployments. Instead, this subsection identifies the trust questions every CPB deployment **MUST** answer in its local trust policy, and provides default guidance.

8.2.1. Who is Authorized to Append CPBs

By default, any node along the bundle's path **MAY** append a CPB. This is permissive and suitable for low-stakes deployments where the cost of bad routing is recoverable, subject to the fragment restrictions in Section 3.2.1. For higher-stakes deployments, the local trust policy **SHOULD** restrict CPB authorship to configured producers and credentials. For authenticated routing use, a receiver **MUST** validate the security source's credential and authorize the authenticated principal for the applicable routing scope and metric. If field 3 is present, the receiver **MUST** additionally establish that this principal is authorized to assert that producer identifier; an explicit delegation in local policy can supply this binding. Field 3 alone **MUST NOT** establish identity or confer a producer's trust rank. If field 3 is absent, authenticated source attribution uses the authenticated principal, or an explicit producer binding supplied by local policy. CPBs that fail these checks **MUST NOT** be used for authenticated routing; their retention or removal follows the applicable BPSec policy. Rejections **SHOULD** be logged using

deployment management facilities informed by [RFC9675] Section 4.3.

8.2.2. Cryptographic Adjudication of Conflicting CPBs

When a bundle carries multiple CPBs from different sources with conflicting probabilities, the eligibility and precedence rules of Section 3.6 apply within each compatible metric, applicability, and normalization profile partition. In trust-policy contexts requiring authentication, a CPB **MUST** pass BIB verification, the coverage checks in Section 8.3, and producer authorization in Section 8.2.1 before entering the precedence calculation. Unsigned, unverifiable, BIB-invalid, or unauthorized CPBs **MUST** be excluded in those contexts. Source trust ranking **MUST** use the authenticated, authorized producer binding when authentication is required.

8.2.3. Downstream Handling of Upstream BIBs

A downstream router that cannot verify an upstream BIB (missing key material, expired credentials, unrecognized security context) follows local BPsec policy. Subject to that policy and the fragment restrictions in Section 3.2.1, CPB routing behavior is as follows:

- * ***Strict:*** Treat the unverifiable CPB as absent for routing decisions. This is **RECOMMENDED** as the default and **REQUIRED** for routing use when authentication is required. Where BPsec policy permits, retain the CPB and its BIB when forwarding so downstream routers with the correct credentials may still consume it.
- * ***Permissive:*** Use the unverified CPB advisably only when local policy permits unauthenticated routing input. Its declared metric-type, applicability, and normalization requirements still apply; the receiver **MUST NOT** relabel it as generic or import its claimed source trust rank. This is **NOT RECOMMENDED** in adversarial environments but may be appropriate in cooperative single-operator deployments where credential rollout lags CPB adoption.

A router **MUST NOT** silently strip an unverifiable BIB-protected CPB and re-sign it with its own credentials. Doing so conceals that the router did not authenticate the upstream producer. On a nonfragment bundle, a router expressing a different estimate **MUST** append a new CPB with its own producer attribution and BIB while preserving the unverifiable original. On a fragment, it **MUST NOT** add a new BIB and follows the local routing fallback in Section 3.2.1.

8.2.4. Route Poisoning by Semi-Trusted Relays

A relay that is trusted enough to participate in the network, but not trusted to make routing decisions for the entire domain, can still degrade routing by producing biased CPB advertisements. Defenses against this attack model are imperfect at the CPB layer.

Mitigations SHOULD include:

- * Plausibility checks: a receiver SHOULD compare an asserted value with compatible local estimates under the same metric, applicability, and normalization profile. Numerical deviation tolerances MUST be defined for that profile. This specification supplies no universal ratio between routing scores and observed delivery frequencies. Claims inconsistent with compatible local evidence SHOULD be discounted or ignored.
- * Source reputation: trust policies MAY maintain per-source reputation scores derived from historical accuracy, and weight CPBs by source reputation when consuming them. CPB itself does not specify a reputation mechanism. Deployments may manage such state within the DTN Management Architecture ([RFC9675]), which does not define a CPB-specific reputation algorithm.
- * Bounded influence (Section 4.4): cap how much any single CPB can shift the local routing decision relative to the router's own observations.

None of these mitigations defeat a sufficiently capable adversary. They raise the cost of route poisoning and make casual misbehavior more detectable. Trust frameworks that fully close this gap (e.g., formal reputation aggregation or attestation chains) are out of scope for this document.

8.2.5. Inter-Domain Trust

When a bundle carrying a CPB crosses an administrative boundary, the receiving domain has no a priori basis for trusting the CPB's producer. Inter-domain trust SHOULD be established via one of the following approaches:

- * A bilateral or multilateral trust agreement that designates which PCE identifiers and BIB credentials from the foreign domain are accepted
- * A gateway router at the domain boundary that consumes the foreign CPB advisably, validates it against local observations, and (optionally) appends a domain-native CPB expressing the gateway's translated assessment

- * A policy of discarding all CPBs at the domain boundary, with the receiving domain regenerating CPBs internally from its own observations

This document does not standardize inter-domain trust establishment. Deployments SHOULD document their inter-domain CPB policy explicitly. This remains an open question.

8.2.6. Replay Resistance

The timestamp (field 2) and validity duration (field 4), combined with the skew tolerances defined in Section 4.1, bound the staleness of estimates when those fields are present. They do not prevent reuse within an accepted time window. BIB protection covers all CPB data, including its timestamp and validity fields, and authenticated routing use MUST also bind the CPB to the containing bundle and its destination as specified in Section 8.3. Merely listing the CPB as a BIB target does not establish that association: a context omitting bundle information can allow an unchanged CPB and BIB to be transplanted to a bundle with a different destination. Bundle binding prevents this substitution, but does not prevent replay of an entire valid bundle. Deployments requiring suppression of such replays SHOULD combine their freshness policy with duplicate-bundle detection.

8.3. Integrity Protection

For deployments in untrusted networks or where source authentication is required, the CPB MUST be protected using Block Integrity Blocks (BIBs) as defined in [RFC9172] (BPsec), using a context that provides cryptographic source authentication with a MAC or digital signature. A receiver MUST verify that protection and apply Section 8.2.1 before using a CPB under such a policy. An absent or unacceptable CPB supplies no routing estimate; local routing state remains available. CPB does not require security operations on a bundle that carries no CPB.

For authenticated routing use, the CPB MUST be listed as a security target of its protecting BIB. That BIB MUST authenticate the complete CPB block-type-specific data, the CPB's block identity and processing flags, and its association with the containing bundle, including the destination. When using BIB-HMAC-SHA2 ([RFC9173] Section 3), senders MUST use integrity scope flags 7 (Section 3.3.3), either explicitly or through that context's default, covering the primary block, target header, and security header in addition to the CPB data. Receivers MUST check that scope before accepting the CPB for authenticated routing, even when its MAC verifies. Other BIB contexts MUST provide equivalent authentication of the CPB data,

containing bundle, and block identities and processing flags. This coverage requirement is additional to identifying the CPB as a BIB target and is subject to the fragment handling in Section 3.2.1.

8.3.1. Source CPB with Append-Only Intermediates

For a nonfragment bundle, the source creates a CPB and attaches a BIB naming that CPB's block number as its target and providing the coverage defined above. Intermediate routers with fresher estimates MAY append new CPBs with their own producer attribution and BIBs. They MUST NOT modify the protected upstream CPB or invalidate its security operation. A router acting only as a security verifier retains a verified upstream BIB operation as required by [RFC9172] Section 5.1.2. Fragments follow Section 3.2.1 instead of this update procedure.

8.3.2. Strip-and-Replace Model

As an alternative on a nonfragment bundle, a forwarder configured as the security acceptor for the affected upstream BIB operation MAY use strip-and-replace. It authenticates and accepts that operation under local policy, removes the operation-specific BIB information as required by [RFC9172] Section 5.1.2, and removes the old CPB. An emptied BIB MUST be removed; operations for unrelated targets MUST be preserved. The node MUST NOT remove the CPB if doing so would invalidate another security operation that it is not authorized to accept. The forwarder then inserts a fresh CPB with its own estimate and producer attribution. It protects the new CPB with its own BIB using the coverage defined above. Verification alone does not confer the security-acceptor role. This update model MUST NOT be applied to fragments. Deployments SHOULD choose one model per administrative domain and document the acceptor and producer-authorization policy.

8.3.3. Prohibition on BCB Encryption

Implementations MUST NOT encrypt the CPB using a Block Confidentiality Block (BCB). Encrypting a routing-critical extension block renders it opaque to the forwarding nodes that need it. Integrity protection (BIB) is appropriate; confidentiality protection (BCB) is not.

8.4. Mitigations

- * MAC or signature verification, bundle-binding checks, and producer authorization before authenticated routing use
- * Timestamp validation (discard old or future-dated CPBs)

- * Secondary validation against compatible local evidence under the applicable metric profile
- * Conservative thresholds for critical payloads

8.5. Privacy Considerations

CPB metadata may reveal information about network topology and operational patterns. Per-path probability entries expose which next-hop nodes are reachable; the source PCE identifier (field 3) claims the identity of the producer. Deployments in adversarial environments SHOULD consider omitting the source PCE identifier when source attribution is not required.

9. IANA Considerations

This document requests IANA to assign a code point from the Bundle Block Types registry ([RFC9171] Section 9.1) under the Specification Required policy ([RFC8126]). This document does not request a specific numeric value.

Value	Description	Reference
TBD	Contact Probability Block	This document

Table 2: Bundle Block Types registration request

Hex dumps and CDDL examples in this document use block type 200 (0xC8). That value is in the RFC 9171 experimental and private-use range (192-255) and is for illustration and laboratory interoperation with those examples only. After IANA assignment, implementations SHOULD parameterize the block-type code so the assigned value can replace 200.

This document also requests IANA to create a new "CPB Metric Types" registry in the Bundle Protocol registries group (not a sub-registry of Bundle Block Types). The registration policy is Specification Required ([RFC8126], Section 4.6). The value space is CBOR unsigned integers, 0 through 18446744073709551615; this document assigns 0-4. Values 5-191 are Unassigned (Specification Required). Values 192-255 are Reserved for Private or Experimental Use; private deployments that mint local metric types MUST use this range. Values 256 through 18446744073709551615 are Unassigned (Specification Required). The registry records the Value, Description, and Reference shown below.

Value	Description	Reference
0	PRoPHET Delivery Predictability	This document
1	CGR Contact Confidence	This document
2	MaxProp Delivery Likelihood	This document
3	RAPID Utility Metric	This document
4	Generic Probability	This document

Table 3: Initial CPB Metric Types registrations

9.1. Guidance for Designated Experts

A registration request MUST provide a short metric description, a stable public specification, and contact information for the registrant. The specification MUST define the meaning and direction of the value, the subjects and time interval to which it applies, default and per-path interpretations, normalization or calibration requirements, and the conditions for comparing values from different producers. It MUST describe use of the existing CPB fields and any additional deployment context needed by a consumer.

Designated experts evaluate whether these definitions permit independent implementations to interpret the metric consistently, preserve the metric-isolation and applicability rules in this document, and avoid conflicting reuse of an assigned value. Requests duplicating an existing metric SHOULD explain why a separate assignment is needed. Experts SHOULD prefer an available value in 5-191 when suitable, but larger values use the same registration policy. Registration does not endorse the accuracy of an estimator or establish routing-performance benefits.

10. Overhead Analysis

CPB size depends on the chosen fields and their encoded lengths. Table 4 gives reproducible examples using binary16 probabilities, block type 200, block number 2, processing flags 0x11, and CRC type 0. These counts exclude any BIB used for authentication under Section 8.3. The canonical-block column includes the array and BTSD byte-string headers. Adding a CRC-16 or CRC-32C value instead adds 3 or 5 bytes, respectively, including the CRC byte-string header.

Configuration A contains field 0 = 0.5. B adds field 2 = 840000000, field 4 = 3600, and field 5 = 1. C also includes eight field-1 entries for uint next hops 100 through 107, each with probability 0.5. D includes all these fields and field 3 = the 17-byte string "producer-node-100", field 6 = 0.75, and field 7 = 1.

Configuration	CPB map bytes	Canonical-block bytes
A: Probability only	5	12
B: Probability and metadata	17	24
C: Eight next hops and metadata	67	75
D: All defined fields, eight next hops	92	100

Table 4: Example CPB Encoded Sizes

Table 5 expresses only these canonical-block bytes as a percentage of payload bytes. It excludes the primary, payload, other extension blocks, convergence-layer framing, and BIB overhead. It is not the total bundle overhead.

Payload (bytes)	A (12 bytes)	B (24 bytes)	C (75 bytes)	D (100 bytes)
50	24.0%	48.0%	150.0%	200.0%
256	4.7%	9.4%	29.3%	39.1%
1024	1.2%	2.3%	7.3%	9.8%
4096	0.3%	0.6%	1.8%	2.4%

Table 5: Example CPB Bytes as a Percentage of Payload Size

Larger identifiers, optional extension fields, additional CPBs, or more path entries increase these costs. There is no universal 32-byte upper bound or payload-independent overhead percentage.

10.1. Computational and Storage Overhead

The table above quantifies only wire-byte overhead. CPB consumption imposes additional costs on forwarders that **SHOULD** be considered when sizing a CPB-enabled deployment, particularly on constrained nodes:

CBOR decode cost Decoding costs depend on the number of entries, identifier lengths, decoder implementation, and floating-point support. The example maps above contain 5-92 bytes; extension values or longer identifiers can require more. This document does not provide measured decoder timings.

BIB verification cost If the CPB is BPSec-protected, authenticated consumption requires verification of its security operation. MAC verification (e.g., HMAC) and public key signature verification (e.g., ECDSA or Ed25519) have different costs that depend on the selected security context and hardware support. In the worst case (resource-constrained node, no hardware crypto), BIB verification can dominate CPB processing time. Implementations **MAY** cache verification results within a single bundle's processing window. A cached result **MUST** be invalidated if any covered bytes, associated primary-block context, credential, or applicable authorization policy changes; block numbers alone are not a sufficient cache identity.

Route recomputation Each new CPB observation potentially triggers route recomputation. The cost depends on the routing algorithm and contact-plan size. A local confidence-weighted cost helper is included for deterministic policy tests; its formula and complexity are not asserted to implement the published CGR-UCoP algorithm. The hysteresis and rate-limiting techniques of Section 4.4 bound the recomputation rate.

Memory pressure from multi-CPB accumulation If the strip-and-replace model (Section 8.3.2) is not used, bundles can accumulate multiple CPBs across hops. Each contributes its actual encoded length plus any associated BIB and implementation bookkeeping. Bundle store memory pressure **SHOULD** be considered when designing CPB production policy. Removal of earlier instances is subject to the authorization and security-operation handling in Section 8.3.2.

Freshness and expiry checks Cached estimates require freshness and expiry checks under Section 4.1 before use. A block's timestamp and TTL permit one expiry comparison; work over individual entries depends on the number of entries. Probability decay, if used, follows the metric's deployment profile. The cache **SHOULD** have a bounded size.

On constrained DTN nodes, CPU and storage costs frequently exceed wire-byte costs in their operational impact. Concrete profiling numbers depend on the BP implementation, the cryptographic algorithms in use, and the deployment topology, and are not provided in this document. Implementations SHOULD publish their measured overhead alongside interoperability reports per [RFC7942].

11. Implementation Status

This section is to be removed before publishing as an RFC.

[Note to RFC Editor: per [RFC7942], this section may be removed before publication.]

This section records the status of implementations of the Contact Probability Block at the time of writing, in accordance with [RFC7942].

Reference Python Implementation

- * *Organization:* Independent (N. Perry)
- * *Repository:* <https://github.com/luckyseoul/draft-perry-dtn-cpb>
(<https://github.com/luckyseoul/draft-perry-dtn-cpb>)
- * *License:* MIT License
- * *Modules:* `cpb.py` (encoder/decoder), `test_cpb.py` (wire-format checks), and `config1_sim.py` (a deterministic local policy-test harness).
- * *Scope:* Encoding and decoding CPB block-type-specific data. Full routing policy, BPv7 block processing, and BPsec integration remain outside this codec.
- * *Maturity:* Reference encoder/decoder and local development helpers. Experimental evaluation is planned as described in Section 12.
- * *Contact:* nick@perrybrandsllc.com

Experimental Scaffolding The repository's `impl` directory retains a local cost helper and an experiment plan for possible future evaluation. No experimental findings are reported here.

No other implementations are known to the author at the time of writing. Reports of new implementations are encouraged.

12. Planned Experimental Evaluation

This unsubmitted Experimental draft proposes the following evaluation plan. These experiments have not been carried out for this draft; no results or performance benefits are asserted.

Routing behavior Compare the same routing implementation with and without CPB consumption under identical contact plans, traffic, resource limits, and random seeds. Include scheduled and uncertain contacts, congestion, and partial CPB deployment. Record delivery ratio, delivery delay, transmission volume, and resource use.

Estimate quality and stability Vary estimate error, age, availability, and update frequency. Examine route changes and fallback behavior with stale, missing, inconsistent, or deliberately misleading estimates. Keep metric applicability and normalization profiles explicit.

Interoperability and security processing Plan exchanges between independent BPv7 implementations covering CPB encoding, unknown fields, multiple blocks, legacy nodes, authentication, authorized updates, and fragmentation.

Method and reporting Specify scenarios, baselines, sample sizes, analysis methods, and acceptance criteria before execution. Publish configurations, implementation versions, data, uncertainty, and unsuccessful outcomes with any later results.

Execution and reporting belong to later work. The reference codec and development checks do not establish the outcomes of this plan.

13. Reference Implementation

The reference implementation supporting this specification consists of the following Python modules under `impl/`:

- * `*cpb.py*` - CBOR encoder/decoder, implementing the CDDL schema in Section 3.2 and the encoding rules in Section 3.4.
- * `*test_cpb.py*` - wire-format conformance suite (encode/decode, hex tables, invalid floats, path-array SHOULD).
- * `*config1_sim.py*` - a deterministic local policy-test harness.
- * `*test_config1_policies.py*` - unit tests for the confidence-weighted cost formula.

- * `*test_sim_cpb_bridge.py*` - bridge tests showing Configuration 1 confidences encode/decode as metric-type 1 CPB maps without changing confidence-weighted route cost ranking.
- * `*test_draft_consistency.py*` - structural checks that draft and documentation claims stay aligned with shipped code.

13.1. Repository

Source code is published under the MIT License at:

<https://github.com/luckyseoul/draft-perry-dtn-cpb>
(<https://github.com/luckyseoul/draft-perry-dtn-cpb>)

13.2. Reference Implementation Development Checks

```
<CODE BEGINS>
git clone https://github.com/luckyseoul/draft-perry-dtn-cpb
cd draft-perry-dtn-cpb
python3 -m pip install -r impl/requirements.txt

python3 impl/test_cpb.py
python3 impl/test_cpb_validation.py
python3 impl/test_config1_policies.py
python3 impl/test_sim_cpb_bridge.py
python3 impl/test_draft_consistency.py
cddl compile-cddl --cddl impl/cpb.cddl
<CODE ENDS>
```

Figure 8: Development Check Commands

These commands are provided for reference-code development. They do not execute the experimental plan in Section 12.

13.3. Relationship to Updates of BPv7

[RFC9171] (BPv7) has been updated by [RFC9713] (administrative record types registry, January 2025) and [RFC9758] (ipn URI scheme updates, May 2025) since this CPB work began. Neither update affects the wire format defined in Section 3:

- * [RFC9713] concerns administrative record types; CPB is an extension block, not an administrative record, so RFC 9713 does not apply directly.

- * [RFC9758] updates the "ipn" URI scheme. CPB implementations MUST encode and decode ipn EIDs per RFC 9758, including Default Allocator FQNNs. The LocalNode ipn URI form (Allocator=0, NodeNumber= $2^{32} - 1$) has no meaning off-node and MUST NOT appear in CPB path-entries transmitted to other nodes.
- * RFC 9171 erratum 8376 (<https://www.rfc-editor.org/errata/eid8376>) records concerns about extension-block placement during fragmentation. It is Held for Document Update and does not replace the published fragmentation rules. CPB's handling under those rules is defined in Section 3.2.1.

Related security work includes the BPsec COSE context [I-D.ietf-dtn-bpsec-cose] (work in progress) and ACME DTN Node ID validation [RFC9891]. These address security contexts and credential provisioning, respectively. Neither independently establishes a credential's authority over a CPB producer identifier or supplies the CPB applicability and bundle-association policy required here.

14. References

14.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.
- [RFC9171] Burleigh, S., Fall, K., and E. Birrane, "Bundle Protocol Version 7", RFC 9171, DOI 10.17487/RFC9171, January 2022, <<https://www.rfc-editor.org/info/rfc9171>>.
- [RFC9172] Birrane, E. and K. McKeever, "Bundle Protocol Security (BPsec)", RFC 9172, DOI 10.17487/RFC9172, January 2022, <<https://www.rfc-editor.org/info/rfc9172>>.
- [RFC9173] Birrane, E., White, A., and S. Heiner, "Default Security Contexts for Bundle Protocol Security (BPsec)", RFC 9173, DOI 10.17487/RFC9173, January 2022, <<https://www.rfc-editor.org/info/rfc9173>>.

- [RFC8949] Bormann, C. and P. Hoffman, "Concise Binary Object Representation (CBOR)", STD 94, RFC 8949, DOI 10.17487/RFC8949, December 2020, <<https://www.rfc-editor.org/info/rfc8949>>.
- [RFC8610] Birkholz, H., Vigano, C., and C. Bormann, "Concise Data Definition Language (CDDL): A Notational Convention to Express Concise Binary Object Representation (CBOR) and JSON Data Structures", RFC 8610, DOI 10.17487/RFC8610, June 2019, <<https://www.rfc-editor.org/info/rfc8610>>.
- [RFC8126] Cotton, M., Leiba, B., and T. Narten, "Guidelines for Writing an IANA Considerations Section in RFCs", BCP 26, RFC 8126, DOI 10.17487/RFC8126, June 2017, <<https://www.rfc-editor.org/info/rfc8126>>.
- [RFC9758] Taylor, R. and E. Birrane, "Updates to the "ipn" URI Scheme", RFC 9758, DOI 10.17487/RFC9758, May 2025, <<https://www.rfc-editor.org/info/rfc9758>>.
- [RFC6693] Lindgren, A., Doria, A., Davies, E., and S. Grasic, "Probabilistic Routing Protocol for Intermittently Connected Networks", RFC 6693, DOI 10.17487/RFC6693, August 2012, <<https://www.rfc-editor.org/info/rfc6693>>.

14.2. Informative References

- [IPNSIG2023] Kaneko, Y., Cerf, V., Tabunshchuk, H., Burleigh, S., Blanchet, M., Scott, K., Suzuki, K., Garcia, O., Chappell, L., Danielson, H., Pace, S., Green, J., and J. Schier, "Solar System Internet Architecture and Governance -- from the Moon to Mars and beyond", September 2023, <https://www.ipnsig.org/_files/ugd/d716c2_5df0d50ff8ab466380dadc75e166cfcf.pdf>.
- [RFC9675] Birrane, E., Heiner, S., and E. Annis, "Delay-Tolerant Networking Management Architecture", RFC 9675, DOI 10.17487/RFC9675, November 2024, <<https://www.rfc-editor.org/info/rfc9675>>.
- [RFC9713] Sipos, B., "Bundle Protocol Version 7 Administrative Record Types Registry", RFC 9713, DOI 10.17487/RFC9713, January 2025, <<https://www.rfc-editor.org/info/rfc9713>>.

- [RFC9891] Sipos, B., "Automated Certificate Management Environment (ACME) Delay-Tolerant Networking (DTN) Node ID Validation Extension", RFC 9891, DOI 10.17487/RFC9891, November 2025, <<https://www.rfc-editor.org/info/rfc9891>>.
- [RFC7942] Sheffer, Y. and A. Farrel, "Improving Awareness of Running Code: The Implementation Status Section", BCP 205, RFC 7942, DOI 10.17487/RFC7942, July 2016, <<https://www.rfc-editor.org/info/rfc7942>>.
- [Raverta21] Raverta, F. D., Fraire, J. A., Madoery, P. G., Demasi, R. A., Finochietto, J. M., and P. R. D'Argenio, "Routing in Delay-Tolerant Networks under uncertain contact plans", Ad Hoc Networks, Volume 123, Article 102663, DOI 10.1016/j.adhoc.2021.102663, 2021, <<https://doi.org/10.1016/j.adhoc.2021.102663>>.
- [Spyropoulos05] Spyropoulos, T., Psounis, K., and C. S. Raghavendra, "Spray and Wait: An Efficient Routing Scheme for Intermittently Connected Mobile Networks", WDTN '05: Proceedings of the 2005 ACM SIGCOMM workshop on Delay-tolerant networking, DOI 10.1145/1080139.1080143, 2005, <<https://doi.org/10.1145/1080139.1080143>>.
- [CGREB] Birrane, E., "Contact Graph Routing Extension Block", Work in Progress (expired), Work in Progress, Internet-Draft, draft-irtf-dtnrg-cgreb-00, October 2013, <<https://datatracker.ietf.org/doc/html/draft-irtf-dtnrg-cgreb-00>>.
- [SABR] CCSDS, CCSDS., "Schedule-Aware Bundle Routing", CCSDS 734.3-B-1, July 2019, <<https://public.ccsds.org/Pubs/734x3b1.pdf>>.
- [CGR] Burleigh, S., "Contact Graph Routing", Work in Progress (expired), Work in Progress, Internet-Draft, draft-burleigh-dtnrg-cgr-01, 8 July 2010, <<https://datatracker.ietf.org/doc/html/draft-burleigh-dtnrg-cgr-01>>.
- [I-D.ietf-dtn-bpsec-cose] Sipos, B., "Bundle Protocol Security (BPsec) COSE Context", Work in Progress, Internet-Draft, draft-ietf-dtn-bpsec-cose-17, 3 August 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-dtn-bpsec-cose-17>>.

[I-D.ietf-dtn-bp-sand]

Sipos, B. and J. Deaton, "Bundle Protocol (BP) Secure Advertisement and Neighborhood Discovery (SAND)", Work in Progress, Internet-Draft, draft-ietf-dtn-bp-sand-03, July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-dtn-bp-sand>>.

[I-D.ietf-tvr-schedule-yang]

Qu, Y., Lindem, A., Kinzie, E., Fedyk, D., and M. Blanchet, "A YANG Data Model for Scheduled Attributes", Work in Progress, Internet-Draft, draft-ietf-tvr-schedule-yang-12, June 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-tvr-schedule-yang>>.

[I-D.taylor-dtn-dpp]

Taylor, R., "DTN Peering Protocol", Work in Progress, Internet-Draft, draft-taylor-dtn-dpp-01, July 2026, <<https://datatracker.ietf.org/doc/html/draft-taylor-dtn-dpp>>.

Appendix A. Mars Mission Scenario

Consider a Mars rover (node 200) communicating with an Earth relay station via an orbiter (node 100):

1. **Initial contact:** Rover creates a critical status bundle with a CPB indicating default probability 0.6 (dusty conditions), metric-type 1 (cgr-confidence), a creation timestamp, and validity_duration 3600s. A shared applicability profile binds the default to the rover-to-orbiter contact identified in the contact plan.
2. **Rover-to-orbiter:** A confidence-weighted consumer using cost = latency / confidence prefers a later, higher-confidence contact when that cost is lower than the dusty path.
3. **Later contact:** Dust clears; the rover appends or replaces the CPB with probability 0.95 and a new timestamp.
4. **Downstream:** Relays do not apply the rover's contact confidence to their own outgoing links. They use local routing state unless a selected CPB supplies values whose applicability profile covers their routing operation, after eligibility and trust/BIB checks.

If the first CPB ($P=0.6$, validity 3600 s) is delayed 10 hours, field 4 has expired. Per Section 4.1, receivers MUST ignore that CPB for routing. This document does not define a residual decay formula after expiry.

Appendix B. Acknowledgements

Brian Sipos provided review comments on CDDL, BTSD embedding, fragmentation pruning, identical-timestamp tie-breaking, the per-path array size limit, scheme-mismatch logging, future-dated timestamp discard, and IANA considerations. Those comments improved this draft. Remaining errors are the author's.

Appendix C. Change Log

This section is to be removed before publishing as an RFC.

[RFC Editor Note: Please remove this entire section before publication.]

C.1. Unsubmitted working draft (September 2026)

- * Initial unsubmitted Experimental working draft (-latest): Contact Probability Block wire format, metric-type registry, operational and security considerations, and IANA requests.
- * CBOR float16 encoding with deterministic encoding rules; format version field (key 7); per-path array size guidance as SHOULD with local enforcement on constrained links.
- * BPsec integrity models (append-only and strip-and-replace); BCB encryption of CPB prohibited for routing use.
- * Relationship to PROPHET, MaxProp, RAPID, Spray-and-Wait, CGR, and DTNMA; IPNSIG Section 4.2 problem framing.
- * Planned experimental evaluation; no experimental results or demonstrated routing-performance improvement.
- * Reference implementation and development check commands (Section 13).
- * Uses the published BPv7 and BPsec fragmentation rules; cites [RFC9713] and [RFC9758] as updates related to BPv7 administration and the ipn scheme.

- * Explicit metric applicability, lifetime and precedence rules; authenticated bundle and producer binding; fragment-safe fallback; strict codec validation and corrected wire examples, size accounting, references, and registry review guidance.
- * Review fixes: finite-float reject policy; trust-before-timestamp precedence for authenticated use; next-hop scheme independent of destination EID; Independent workgroup line; MaxProp delivery likelihood naming; Metric Types private-use range 192-255.
- * Pre-submit polish: valid BCP 14 wording for path-entry schemes; IANA MaxProp description aligned with delivery-likelihood; Intro/Motivation claim hygiene; RFC6693 normative for metric-type 0 aging; terminology/notation cleanup; reproduce listing includes validation tests; Fig2 hex wrap; CDDL comment sync.
- * Pre-Datatracker packaging: keep docname draft-perry-dtn-cpb-latest; document date 2026-09-07; regenerated txt/html/pdf.

Author's Address

Nicholas Perry
Independent
Council Bluffs, Iowa
United States of America
Email: nick@perrybrandsllc.com